BITMANIP (Zba/Zbb): using the RISC-V bit-manipulation extensions

Date: 2026-11-11. Board: VisionFive 2 (JH7110, 4x SiFive U74, ISA `rv64imafdc_..._zba_zbb`), OpenSSL 3.5.9. Compared with the builds used for all earlier results ("plain", compiled for `rv64gc` without the extensions) and with the **BASELINE** (`baseline_REPORT_default_clock.md`).

The idea

The U74 cores implement **Zba** (address generation: `sh1add`/`sh2add`/`sh3add`) and **Zbb** (basic bit-manipulation: rotations `rol`/`ror`/`rori`, byte-reverse `rev8`, `andn`/`orn`/`xnor`, `clz`/`ctz`/`cpop`, `min`/`max`). All earlier OpenSSL builds targeted plain `rv64gc` and never used them. The biggest expected winner is **Keccak (SHA-3/SHAKE)**, which is mostly 64-bit rotations and XORs and is used heavily inside **ML-KEM** and **ML-DSA**.

No source code was changed; only a compiler flag:

Build	Location	Configure options
std ("plain")	`/opt/openssl-3.5`	`shared no-tests no-docs`
opt ("plain")	`~/openssl-3.5-opt`	+ `enable-ec_nistp_64_gcc_128` (fast P-256)
bitmanip	`~/openssl-3.5-bitmanip`	+ `enable-ec_nistp_64_gcc_128` **`-march=rv64gc_zba_zbb`**

Check that GCC 12.2 really emits the instructions (test function, `-O2`):

C code	`-march=rv64gc`	`-march=rv64gc_zba_zbb`
64-bit rotate `(x<<13)|(x>>51)`	`sll` + `srl` + `or` (3 instructions)	`rol` (1 instruction)
array index `p[i]` (8-byte elements)	`sll` + `add`	`sh3add`

Part 1: crypto micro-benchmarks at a fixed 1.5 GHz clock

File: `bitmanip_micro_fixed1500.txt`. Two full passes; pass 1 and pass 2 agree within ~1 %. The CPU was fixed at 1.5 GHz (`performance`) for ~8 minutes only for this measurement and then restored to the default. (An earlier attempt under the default `ondemand` governor gave inconsistent numbers, e.g. identical SHA-256 code differing 2x between builds, so it's kept only as `bitmanip_micro_ondemand_UNRELIABLE.txt` and not used.)

Pass 1 values; "plain" = the opt build (identical to std for everything except P-256).

Operation	Plain (rv64gc)	Bitmanip (Zba/Zbb)	Change
SHA3-256 throughput	32.8 MB/s	**49.1 MB/s**	**+50 %**
SHAKE-128 throughput	37.6 MB/s	**56.7 MB/s**	**+51 %**
SHA-256 throughput	31.6 MB/s	**44.5 MB/s**	**+41 %**
ML-KEM-768 keygen	0.454 ms	**0.367 ms**	**–19 %**
ML-KEM-768 encaps	0.356 ms	**0.309 ms**	**–13 %**
ML-KEM-768 decaps	0.530 ms	**0.455 ms**	**–14 %**
X25519MLKEM768 keygen (client)	0.715 ms	**0.630 ms**	**–12 %**
X25519MLKEM768 decaps (client)	0.823 ms	**0.749 ms**	**–9 %**
ML-DSA-44 verify	0.987 ms	**0.836 ms**	**–15 %**
ML-DSA-65 verify	1.578 ms	**1.319 ms**	**–16 %**
ML-DSA-44 sign	4.40–4.80 ms	4.06–4.26 ms	≈ –11 % (signing has a random retry loop, so it's noisier)
ECDSA P-256 verify	1.413 ms	1.427 ms	no change
X25519 (op/s)	3716	3712	no change
RSA-3072 sign / verify	34.06 / 0.643 ms	34.26 / 0.643 ms	no change

Per hybrid handshake on the board (client side, 1.5 GHz): X25519MLKEM768 keygen + decaps 1.538 → 1.379 ms (–0.16 ms) plus ML-DSA-44 verify –0.15 ms gives about **–0.3 ms of CPU time per handshake**. The legacy handshake (X25519 + RSA verify) gets essentially nothing from the extensions.

Part 2: end-to-end, 1000 runs per mode, default CPU setting

Files: `bitmanip_e2e_raw.csv` (`config,scenario,mode,block,run,ms`), `bitmanip_e2e.log`, `bitmanip_e2e_stats.txt`. Method: 20 blocks x 50 runs; in every block both builds were measured back to back (order alternating) for both modes and both scenarios, so they saw the same network conditions. CPU at the default (`ondemand`, min 312.5 MHz), the same as the baseline. 8000 connections, 0 failures; every hybrid block verified (X25519MLKEM768 negotiated, ML-DSA-44 verified).

Own server (LAN), end-to-end (hybrid = X25519MLKEM768 + ECDSA P-256 + ML-DSA-44)

Build	Mode	Best	Median	Mean	95th pct	Worst
BASELINE (earlier run, plain)	Legacy	14.77	**21.61**	21.53	24.35	31.30 ms
BASELINE (earlier run, plain)	Hybrid	20.51	**27.95**	27.99	31.58	40.05 ms
Plain (this run)	Legacy	15.31	**21.95**	21.92	24.39	31.11 ms
Plain (this run)	Hybrid	20.06	**28.15**	28.35	31.88	43.09 ms
Bitmanip	Legacy	11.68	**21.14**	21.20	24.32	33.28 ms
Bitmanip	Hybrid	19.19	**26.57**	26.96	30.58	38.39 ms

- Hybrid, bitmanip vs plain: **–1.58 ms (–5.6 %)**; per block –1.48 ms, **bitmanip faster in 20 of 20 blocks**.
- Legacy, bitmanip vs plain: –0.81 ms (–3.7 %); bitmanip faster in 15 of 20 blocks.
- Hybrid – legacy gap: plain +6.55 ms → bitmanip +5.46 ms.

Hackathon server (IPv4), TLS handshake

Build	Mode	Best	Median	Mean	95th pct	Worst
BASELINE (earlier run)	Legacy	31.61	**39.92**	45.07	70.84	524.45 ms
BASELINE (earlier run)	Hybrid	36.33	**45.08**	50.09	76.03	359.32 ms
Plain (this run)	Legacy	27.40	**41.59**	51.28	107.76	255.94 ms
Plain (this run)	Hybrid	36.41	**46.69**	56.50	107.06	449.24 ms
Bitmanip	Legacy	31.04	**41.29**	48.99	87.79	274.56 ms
Bitmanip	Hybrid	34.03	**47.46**	54.22	90.80	547.64 ms

- Hybrid, bitmanip vs plain: overall median +0.77 ms, but per block −1.11 ms with bitmanip faster in 13 of 20 blocks. The per-block spread (−49 to +15 ms) is network noise, much larger than the ~1 ms effect: **no measurable difference**.

Findings

1. **The extensions work and help exactly where expected:** SHA-3/SHAKE +50 %, SHA-256 +41 %, ML-KEM −13 to −19 %, ML-DSA verify −15 %. Classical algorithms (RSA, ECDSA, X25519) don't benefit. **Post-quantum crypto benefits more from Zba/Zbb than classical crypto**, which favours hybrid.
2. **End-to-end the effect is real but small:** −1.6 ms (−5.6 %) for full hybrid on the LAN at the default clock, consistent in 20/20 blocks. Against the internet server, network variation hides it.
3. **It doesn't meet the latency target on its own:** at the default clock hybrid (26.57 ms) is still slower than the legacy baseline (21.61 ms). For comparison, raising the minimum CPU clock to 750 MHz (`dynamic_cpu_freq_REPORT.md`) gave −11.6 ms on the same test. **The clock policy matters about 7x more than the instruction-set extensions.**
4. **Cost: zero runtime cost, one build flag.** No code changes, no extra heat or power. The trade-off is portability: the library only runs on CPUs with Zba and Zbb (it would crash on a plain RV64GC core).
5. **Next step (not measured yet):** combine both: bitmanip build + ondemand min 750 MHz. The two effects are independent (fewer instructions vs higher clock), so the combination should be the fastest low-heat configuration.

Summary table vs the baseline (own server, LAN, end-to-end median)

Configuration	Legacy	Hybrid	Hybrid vs baseline legacy (21.61 ms)
BASELINE: plain build, default clock	21.61 ms	27.95 ms	+6.34 ms ❌
Bitmanip build, default clock	21.14 ms	26.57 ms	+4.96 ms ❌
Plain build, ondemand min 750 MHz (dynamic CPU freq)	15.70 ms	16.19 ms	−5.42 ms ✅
Bitmanip build + min 750 MHz	not measured	not measured	expected best

Source: bitmanip_REPORT.md